

Homework 07 — STAT 432 (Fall 2026)

Name: Owen Park **NetID:** owenp3

Everything is in Python (numpy 1.26, pandas 2.2, scikit-learn 1.6). The one random seed used in this report is **4327**, passed to `numpy.random.default_rng` in Question 3 and to the train/test split and the cross-validation folds in Question 4. Questions 1 and 2 have no randomness. The predictor coding for `SAheart` follows the assignment: `famhist` is 1 for `Absent` and 2 for `Present`, and the other eight columns are used as they are.

Question 1: LDA

I estimate the three LDA components by hand with the pooled denominator $n - K$, compute the score of every observation for both classes, and then compare the fitted labels with

```
LinearDiscriminantAnalysis(solver="svd").
```

```
In [1]: import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
pd.set_option("display.width", 130)
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA, QuadraticDiscriminantAnalysis as QDA

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy(dtype=float)
y = heart["chd"].to_numpy()
names = list(heart.drop(columns="chd").columns)
n, p = X.shape
classes = np.array([0, 1]); K = len(classes)

pi_hat = np.array([(y == k).mean() for k in classes]) # (K,)
mu_hat = np.vstack([X[y == k].mean(axis=0) for k in classes]) # (K, p)
Sigma_hat = sum((X[y == k] - mu_hat[i]).T @ (X[y == k] - mu_hat[i]) for i, k in enumerate(classes)) / (n - K) # (p, p), denominator n - K
print(f"n = {n}, p = {p}, K = {K}")
print("prior probabilities:", dict(zip(classes, pi_hat.round(4))))
print("class means (rows = chd 0, 1):")
print(pd.DataFrame(mu_hat, index=classes, columns=names).round(3).to_string())
print("pooled covariance (denominator n - K):")
print(pd.DataFrame(Sigma_hat, index=names, columns=names).round(2).to_string())

n = 462, p = 9, K = 2
prior probabilities: {0: 0.6537, 1: 0.3463}
class means (rows = chd 0, 1):
   sbp  tobacco  ldl  adiposity  famhist  typea  obesity  alcohol  age
0  135.460    2.635  4.344    23.969    1.318  52.368   25.737   15.931  38.854
1  143.738    5.525  5.488    28.120    1.600  54.494   26.623   19.145  50.294
pooled covariance (denominator n - K):
   sbp  tobacco  ldl  adiposity  famhist  typea  obesity  alcohol  age
sbp    405.43   14.59  4.58    49.16    0.34 -15.59   18.94   64.40   95.13
tobacco  14.59   19.24  0.76     7.54    0.02  -2.06    1.83   20.52   22.77
ldl       4.58    0.76  4.00     6.03    0.09  0.34    2.66   -2.53    6.48
adiposity 49.16    7.54  6.03    56.75    0.43  -5.31   22.71   16.12   60.51
famhist   0.34    0.02  0.09     0.43    0.23  0.08    0.18    0.77    1.00
typea   -15.59   -2.06  0.34    -5.31    0.08  95.57    2.64    7.96   -20.28
obesity   18.94    1.83  2.66    22.71    0.18  2.64   17.62    4.69   15.70
alcohol   64.40   20.52 -2.53    16.12    0.77  7.96    4.69   598.28   27.89
age       95.13   22.77  6.48    60.51    1.00 -20.28   15.70   27.89   184.13

In [2]: Sinv = np.linalg.inv(Sigma_hat)
# score[i, k] = x_i' S^-1 mu_k - 0.5 mu_k' S^-1 mu_k + log pi_k
linear = X @ Sinv @ mu_hat.T # (n, K)
quad = -0.5 * np.einsum("kj,jl,kl->k", mu_hat, Sinv, mu_hat) # (K,)
score = linear + quad + np.log(pi_hat) # (n, K)
yhat_manual = classes[score.argmax(axis=1)]
print("shapes: X", X.shape, " Sigma^-1", Sinv.shape, " mu", mu_hat.shape, " scores", score.shape, " yhat", yhat_manual.shape)
cm_manual = pd.crosstab(pd.Series(y, name="observed"), pd.Series(yhat_manual, name="predicted"))
print("in-sample confusion matrix (hand-computed scores):"); print(cm_manual.to_string())
print(f"in-sample misclassification rate: {(yhat_manual != y).mean():.4f}")
```

```

shapes: X (462, 9) Sigma^-1 (9, 9) mu (2, 9) scores (462, 2) yhat (462,)
in-sample confusion matrix (hand-computed scores):
predicted    0    1
observed
0           258  44
1            73  87
in-sample misclassification rate: 0.2532

```

(a) The prior probabilities are 0.654 for `chd = 0` and 0.346 for `chd = 1`, which is just 302/462 and 160/462. Every class mean is higher in the disease group, and the largest gaps are in `age` (38.9 versus 50.3), `tobacco` (2.6 versus 5.5) and `adiposity` (24.0 versus 28.1). The pooled covariance is the sum of the two within-class scatter matrices divided by $n - K = 460$, so it is a 9 by 9 matrix whose diagonal holds the pooled variances; `alcohol` and `sbp` have by far the largest ones because they are measured on the widest scales.

(b) The dimensions line up: $\mathbf{X}$ is 462 by 9, $\hat{\Sigma}^{-1}$ is 9 by 9 and the class means are 2 by 9, so the score matrix is 462 by 2 and the fitted label vector has 462 entries. The in-sample confusion matrix is 258 and 44 in the no-disease row and 73 and 87 in the disease row, which is a training error of 25.3 percent. LDA gets most of the healthy patients right but misses 73 of the 160 disease cases, because the prior for `chd = 1` is small and the two groups overlap heavily on these predictors.

```

In [3]: lda_sk = LDA(solver="svd").fit(X, y) # priors default to the empirical class frequencies
yhat_sk = lda_sk.predict(X)
cm_sk = pd.crosstab(pd.Series(y, name="observed"), pd.Series(yhat_sk, name="predicted"))
print("sklearn confusion matrix:"); print(cm_sk.to_string())
print("sklearn priors:", lda_sk.priors_.round(4), "class means equal to mine:", np.allclose(lda_sk.means_, mu_hat))
print("fitted labels identical to the hand-computed ones:", np.array_equal(yhat_sk, yhat_manual))

sklearn confusion matrix:
predicted    0    1
observed
0           258  44
1            73  87
sklearn priors: [0.6537 0.3463] class means equal to mine: True
fitted labels identical to the hand-computed ones: True

```

(c) The results match exactly. `LinearDiscriminantAnalysis` with the SVD solver and empirical priors gives the same 258 / 44 / 73 / 87 confusion matrix, its stored priors and class means equal mine, and all 462 fitted labels are identical. scikit-learn scales the pooled covariance by $1/n$ internally rather than $1/(n - K)$, but that constant multiplies every score in the same way and cancels in the comparison between classes, so the labels do not change.

Question 2: QDA and marginal screening

The full `zip.train` (7291 images) is the training set and `zip.test` (2007 images) is the test set, all ten digits. I rank the 256 pixels by their variance across the training images, keep the 40 (then 10) with the largest variance, and fit

`QuadraticDiscriminantAnalysis(tol=1e-8)` on those raw pixel values.

```

In [4]: zip_tr = pd.read_csv("data/zip-train.csv.gz", header=None).to_numpy()
zip_te = pd.read_csv("data/zip-test.csv.gz", header=None).to_numpy()
y_tr, X_tr = zip_tr[:, 0].astype(int), zip_tr[:, 1:]
y_te, X_te = zip_te[:, 0].astype(int), zip_te[:, 1:]
print("train", X_tr.shape, " test", X_te.shape, " training images per digit:", np.bincount(y_tr).tolist())

marg_var = X_tr.var(axis=0, ddof=1)
order = np.argsort(-marg_var)
print("largest marginal variances:", marg_var[order[:5]].round(3), " smallest:", marg_var.min().round(4))

def qda_screen(m):
    idx = order[:m]
    qda = QDA(tol=1e-8).fit(X_tr[:, idx], y_tr)
    pred = qda.predict(X_te[:, idx])
    cm = pd.crosstab(pd.Series(y_te, name="observed"), pd.Series(pred, name="predicted"))
    return idx, cm, (pred != y_te).mean(), (qda.predict(X_tr[:, idx]) != y_tr).mean()

```

```
idx40, cm40, err40, train40 = qda_screen(40)
print(f"\nQDA on the 40 highest-variance pixels: test misclassification {err40:.4f} ({int(round(err40 * len(y_te)))})")
print("selected pixels sit in image rows", sorted(set((idx40 // 16).tolist())), "and columns", sorted(set((idx40 % 16).tolist())))
print(cm40.to_string())
```

```
train (7291, 256) test (2007, 256) training images per digit: [1194, 1005, 731, 658, 652, 556, 664, 645, 542, 644]
largest marginal variances: [0.8 0.779 0.776 0.776 0.77 ] smallest: 0.0022
```

```
QDA on the 40 highest-variance pixels: test misclassification 0.1555 (312 of 2007), training 0.0834
selected pixels sit in image rows [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14] and columns [4, 5, 7, 8, 9, 10, 11]
```

predicted \ observed	0	1	2	3	4	5	6	7	8	9
0	319	0	24	6	2	2	1	0	5	0
1	0	238	5	0	11	0	7	0	3	0
2	8	0	168	8	8	3	0	0	3	0
3	5	0	4	128	0	14	0	0	15	0
4	0	2	13	1	150	1	0	1	7	25
5	12	0	3	5	1	131	0	0	6	2
6	0	0	8	0	1	8	148	0	5	0
7	1	0	6	1	6	0	0	124	2	7
8	1	0	10	12	3	2	0	0	137	1
9	0	0	1	2	7	1	1	4	9	152

```
In [5]: idx10, cm10, err10, train10 = qda_screen(10)
print(f"\nQDA on the 10 highest-variance pixels: test misclassification {err10:.4f} ({int(round(err10 * len(y_te)))})")
print(cm10.to_string())
lda40 = LDA().fit(X_tr[:, idx40], y_tr); lda10 = LDA().fit(X_tr[:, idx10], y_tr)
print(f"for reference, LDA on the same pixels: 40 -> {(lda40.predict(X_te[:, idx40]) != y_te).mean():.4f}, 10 -> {(lda10.predict(X_te[:, idx10]) != y_te).mean():.4f}")
min_sv = {m: min(np.linalg.svd(np.cov(X_tr[y_tr == k][:, idx40]), compute_uv=False).min() for k in range(10))}
print("smallest singular value of any class covariance: 40 pixels", f"{min_sv[40]:.2e}", "10 pixels", f"{min_sv[10]:.2e}")
```

```
QDA on the 10 highest-variance pixels: test misclassification 0.3423 (687 of 2007), training 0.2888
```

predicted \ observed	0	1	2	3	4	5	6	7	8	9
0	298	0	11	1	1	16	28	4	0	0
1	0	241	10	0	3	1	1	0	2	6
2	19	2	95	14	14	3	22	21	5	3
3	5	0	6	103	3	42	0	0	7	0
4	1	5	14	0	95	2	4	15	17	47
5	15	0	6	25	4	96	4	2	8	0
6	41	0	15	4	0	18	78	7	5	2
7	4	1	6	0	6	0	1	122	0	7
8	1	2	11	36	12	11	4	1	74	14
9	0	2	9	2	20	0	0	16	10	118

```
for reference, LDA on the same pixels: 40 -> 0.2337, 10 -> 0.3906
smallest singular value of any class covariance: 40 pixels 2.67e-07 10 pixels 2.39e-03
```

(a) With the 40 highest-variance pixels, QDA has a test misclassification rate of **0.1555** (312 of 2007 test images wrong; training error 0.083). The confusion matrix shows the familiar digit confusions: 4 is sent to 9 twenty-five times, 0 to 2 twenty-four times, and 3 to 5 and 8 about fifteen times each, while 1 and 0 are almost always right.

(b) The method works in the sense that the fit now exists and is usable. The 40 selected pixels all sit in the middle columns of the image (columns 4 to 11), where strokes actually vary, so every class has 542 or more training images in a 40-dimensional space and none of the ten class covariance matrices is singular (the smallest singular value, 2.7×10^{-7} , is above the 1×10^{-8} tolerance). The lecture problem was the opposite: with all 256 pixels the border pixels are constant within a class and the 10 by 10 covariances cannot be inverted. It works only moderately well, though. The separate covariances do earn their keep, since LDA on the same 40 pixels is at 0.234, but 15.6 percent is far from what a nearest-neighbour rule reaches on this data, because screening by marginal variance keeps the pixels that vary most across all digits, which is not the same as the pixels that separate the digits.

(c) With only 10 pixels the test error more than doubles to **0.3423** (687 wrong). Ten pixel values simply do not carry enough information to tell ten digits apart: digits that share a central vertical stroke (4, 7, 9, 1) or a central loop (0, 6, 8, 3, 5) land on each other, which is exactly where the big off-diagonal counts are. Going from 40 to 10 pixels removes 30 predictors that QDA could use, so the bias from the missing information grows much faster than the small amount of variance saved by estimating smaller covariance matrices. The training error (0.289) is almost as high as the test error, so this is underfitting, not overfitting.

Question 3: Compare LDA and QDA

Two classes, $p = 10$. The first two covariates come from the bivariate normals in the assignment with correlation $+\rho$ in class 1 and $-\rho$ in class 2; the other eight are independent standard normals in both classes. For each ρ I generate 100 training and 1,000 test observations per class, fit LDA (one pooled covariance) and QDA (one covariance per class) with equal priors on all 10 covariates, and record the test misclassification rate. Seed 4327.

In [6]:

```
rng = np.random.default_rng(4327)
rhos = [0, 0.2, 0.4, 0.6, 0.8, 0.9]

def generate(rho, n_per_class):
    C1 = np.array([[1, rho], [rho, 1]]); C2 = np.array([[1, -rho], [-rho, 1]])
    X1 = np.hstack([rng.multivariate_normal([-1, 0], C1, n_per_class), rng.standard_normal((n_per_class, 8))])
    X2 = np.hstack([rng.multivariate_normal([1, 0], C2, n_per_class), rng.standard_normal((n_per_class, 8))])
    return np.vstack([X1, X2]), np.r_[np.ones(n_per_class, int), 2 * np.ones(n_per_class, int)]

def test_errors(Xa, ya, Xb, yb, cols):
    e_lda = (LDA(priors=[0.5, 0.5]).fit(Xa[:, cols], ya).predict(Xb[:, cols]) != yb).mean()
    e_qda = (QDA(priors=[0.5, 0.5]).fit(Xa[:, cols], ya).predict(Xb[:, cols]) != yb).mean()
    return e_lda, e_qda
```

```
single = []
for rho in rhos:
    Xa, ya = generate(rho, 100); Xb, yb = generate(rho, 1000)
    single.append(test_errors(Xa, ya, Xb, yb, slice(None)))
single = pd.DataFrame(single, index=pd.Index(rhos, name="rho"), columns=["LDA", "QDA"])
print("(a) test misclassification rate, one dataset per rho (same train and test for both methods):")
print(single.round(4).T.to_string())
```

(a) test misclassification rate, one dataset per rho (same train and test for both methods):

rho	0.0	0.2	0.4	0.6	0.8	0.9
LDA	0.1775	0.1745	0.186	0.178	0.1605	0.1605
QDA	0.2235	0.1905	0.192	0.188	0.1325	0.1010

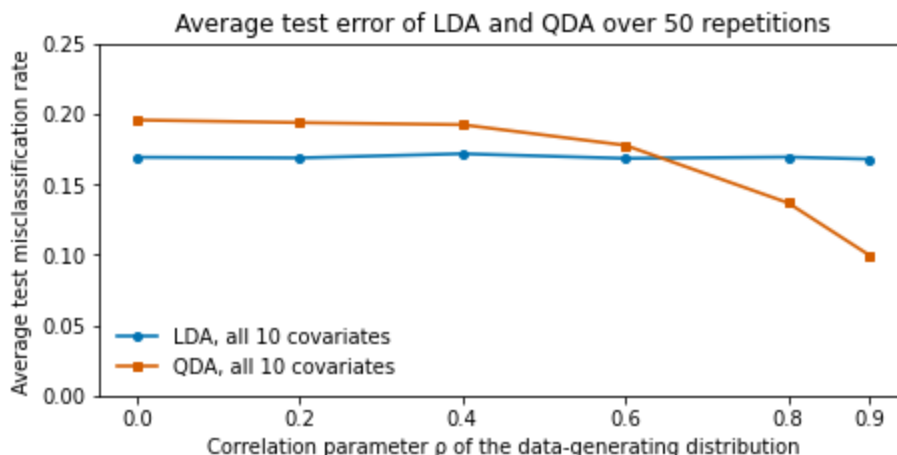
In [7]:

```
R = 50
avg10 = np.zeros((len(rhos), 2)); avg2 = np.zeros((len(rhos), 2))
for i, rho in enumerate(rhos):
    for r in range(R):
        Xa, ya = generate(rho, 100); Xb, yb = generate(rho, 1000)
        avg10[i] += test_errors(Xa, ya, Xb, yb, slice(None)) # all 10 covariates
        avg2[i] += test_errors(Xa, ya, Xb, yb, slice(0, 2)) # only the two informative covariates, same data
avg10 /= R; avg2 /= R
avg = pd.DataFrame(np.hstack([avg10, avg2]), index=pd.Index(rhos, name="rho"),
                  columns=["LDA (10 cov.)", "QDA (10 cov.)", "LDA (2 cov.)", "QDA (2 cov.)"])
print(f"(b) average test error over {R} repetitions; the 2-covariate columns reuse the same datasets and are for part (c):")
print(avg.round(4).T.to_string())

fig, ax = plt.subplots(figsize=(6.5, 3.4))
ax.plot(rhos, avg10[:, 0], color="#0072B2", marker="o", markersize=4, linewidth=1.6, label="LDA, all 10 covariates")
ax.plot(rhos, avg10[:, 1], color="#D55E00", marker="s", markersize=4, linewidth=1.6, label="QDA, all 10 covariates")
ax.set(xlabel="Correlation parameter ρ of the data-generating distribution", ylabel="Average test misclassification rate")
ax.set(title="Average test error of LDA and QDA over 50 repetitions", xticks=rhos, ylim=(0, 0.25))
ax.legend(loc="lower left", frameon=False); plt.tight_layout(); plt.show()
```

(b) average test error over 50 repetitions; the 2-covariate columns reuse the same datasets and are for part (c):

rho	0.0	0.2	0.4	0.6	0.8	0.9
LDA (10 cov.)	0.1691	0.1686	0.1716	0.1683	0.1692	0.1678
QDA (10 cov.)	0.1955	0.1937	0.1922	0.1776	0.1367	0.0995
LDA (2 cov.)	0.1602	0.1588	0.1614	0.1594	0.1599	0.1579
QDA (2 cov.)	0.1601	0.1598	0.1630	0.1515	0.1178	0.0847



(c) At small ρ LDA wins and at large ρ QDA wins, and the crossover is between 0.6 and 0.8. Averaged over 50 repetitions, LDA sits at about 0.17 for every ρ , while QDA starts at 0.196 at $\rho = 0$, is still slightly behind LDA at 0.6 (0.178 versus 0.168), and then drops to 0.137 at 0.8 and 0.099 at 0.9.

The pattern follows from what the two methods assume. At $\rho = 0$ the two classes really do share one covariance matrix, so LDA is the correct model and its linear boundary $x_1 = 0$ is the Bayes rule; QDA fits the same boundary in expectation but estimates two separate 10 by 10 covariances (55 free entries each) from 100 observations per class, and that extra variance costs it about three percentage points. As ρ grows the class ellipses tilt in opposite directions. The pooled covariance LDA estimates is still close to the identity because the $+\rho$ and $-\rho$ average out, so the best linear rule is unchanged and its error stays near $\Phi(-1) \approx 0.16$ no matter how large ρ gets. That is pure bias: the true boundary is a curve, and the more the ellipses tilt the more a straight line misses. QDA can bend its boundary along the tilt, so its bias disappears and its variance cost is more than repaid once ρ is large.

The eight noise covariates hurt both methods even though they carry no information, and they hurt QDA more. Refitting on the same datasets with only X_1, X_2 (the 2-covariate columns in the table) lowers LDA from about 0.169 to 0.160 at every ρ , while at $\rho = 0$ QDA falls from 0.196 to 0.160, which is the same as LDA. Each noise covariate adds entries to every mean vector and covariance matrix that must be estimated, and the estimated discriminant then puts small random weights on those directions, which pushes test points across the boundary by chance. LDA adds 8 means per class and 8 rows to one pooled matrix; QDA adds them to two matrices, each estimated from half the data, so the noise inflates its variance roughly twice as much.

Question 4: Create and use an LDA/QDA analysis skill

(a) **The skill.** My skill is a single file, `.claude/skills/lda-qda-report/SKILL.md` in my course repository (the folder is git-ignored, so the complete text is appended at the end of this report). It is a Claude Code skill. The frontmatter sets `disable-model-invocation: true`, which means Claude never loads it on its own and it is only read when I type `/lda-qda-report ...` myself; the arguments after the slash command reach the skill as `$ARGUMENTS`. The skill fixes the agent's behaviour (seed first, test set touched once, preprocessing fitted inside each fold, only LDA/QDA-based classifiers), the candidate set (LDA, QDA, PCA + LDA and PCA + QDA for every number of components, and a posterior blend of LDA and QDA), the selection rule, and a seven-part one-page report with one figure and one table. Nothing in it mentions heart disease, column names, or any result.

(b) **Applying it.** The prompt I used, in a Claude Code session opened in the course repository, was:

```
/lda-qda-report data=homework/week-07/data/saheart.csv response=chd
predictors="all other columns, famhist coded Absent=1 Present=2"
seed=4327 test_frac=0.25 folds=10. Seek the best misclassification
```

rate within the methods the skill allows and keep plain LDA and plain QDA as comparisons.

The code below is what the agent ran. The stratified 75/25 split is made before any model is fitted, the same ten stratified folds are used for all 21 candidates, standardizing and PCA are fitted on the training part of each fold only, and the test set is used exactly once, for the selected model.

```
In [8]: from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import sklearn

SEED = 4327
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y, random_state=SEED)
folds = list(StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED).split(X_train, y_train))
print(f"train n = {len(y_train)} (chd=1 share {y_train.mean():.3f}), test n = {len(y_test)} (chd=1 share {y_test.me

def posterior(cand, Xa, ya, Xb):
    """P(y = 1) on Xb for candidate `cand`, with every preprocessing step fitted on (Xa, ya) only."""
    if cand.startswith("pca"):
        kind, k = cand[3:6], int(cand.split("-")[1])
        sc = StandardScaler().fit(Xa); pca = PCA(n_components=k).fit(sc.transform(Xa))
        Za, Zb = pca.transform(sc.transform(Xa)), pca.transform(sc.transform(Xb))
        return (LDA() if kind == "lda" else QDA()).fit(Za, ya).predict_proba(Zb)[:, 1]
    if cand.startswith("blend"):
        w = float(cand.split("-")[1])
        return w * LDA().fit(Xa, ya).predict_proba(Xb)[:, 1] + (1 - w) * QDA().fit(Xa, ya).predict_proba(Xb)[:, 1]
    return (LDA() if cand == "lda" else QDA()).fit(Xa, ya).predict_proba(Xb)[:, 1]

cands = ["lda", "qda"] + [f"pcalda-{k}" for k in range(1, p)] + [f"pcaqda-{k}" for k in range(1, p)] + [f"blend-{w}"
cv = {}
for c in cands:
    errs = [(posterior(c, X_train[a], y_train[a], X_train[b]) > 0.5).astype(int) != y_train[b]].mean() for a, b in
    cv[c] = (np.mean(errs), np.std(errs, ddof=1) / np.sqrt(len(errs)))
cv = pd.DataFrame(cv, index=["cv_error", "se"]).T.sort_values("cv_error")
best = cv.index[0]
near = cv[cv.cv_error <= cv.cv_error.iloc[0] + 0.001]
simplicity = lambda c: (0 if c == "lda" else 1 if c == "qda" else 2 + int(c.split("-")[1]) if c.startswith("pcalda"
best = min(near.index, key=simplicity)
print("CV misclassification rate and fold SE, all candidates (best to worst):")
for j in range(0, len(cv), 7): print(cv.iloc[j:j + 7].round(4).T.to_string())
print("selected:", best)

# refit the selected preprocessing + model on all training rows, evaluate once on the test set
p_test = posterior(best, X_train, y_train, X_test)
test_err = ((p_test > 0.5).astype(int) != y_test).mean()
n_t, z = len(y_test), 1.96
centre = (test_err + z**2 / (2 * n_t)) / (1 + z**2 / n_t); half = z * np.sqrt(test_err * (1 - test_err) / n_t + z**
print(f"test misclassification rate of {best}: {test_err:.4f} (95% Wilson interval {centre - half:.3f} to {centre
k_best = int(best.split("-")[1]); sc_f = StandardScaler().fit(X_train); pca_f = PCA(n_components=k_best).fit(sc_f.t
final = LDA().fit(pca_f.transform(sc_f.transform(X_train)), y_train)
print("variance explained by the kept components:", pca_f.explained_variance_ratio_.round(3), " total", pca_f.expla
print("priors:", final.priors_.round(4)); print("class means on the component scale (rows chd 0, 1):"); print(np.ro

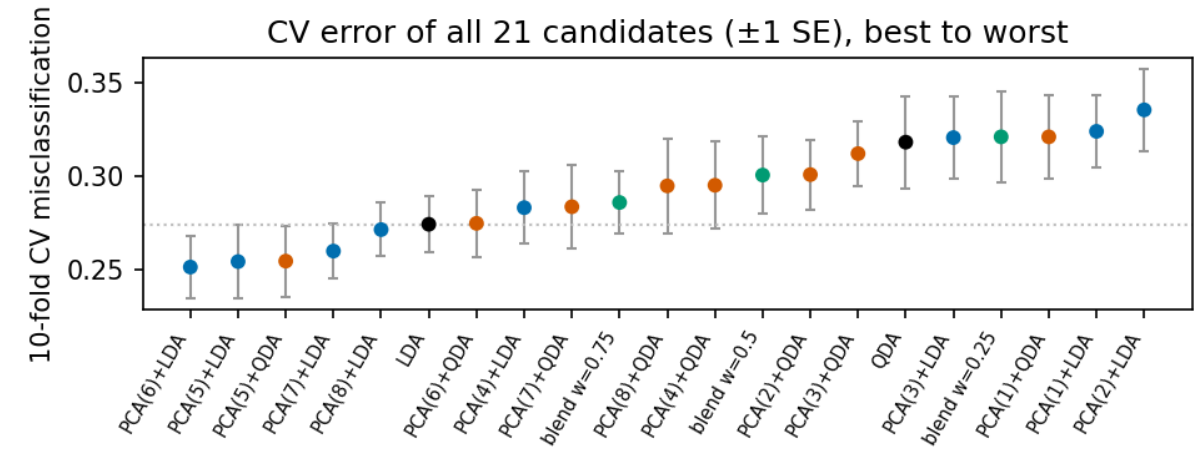
train n = 346 (chd=1 share 0.347), test n = 116 (chd=1 share 0.345), seed 4327, sklearn 1.6.0
CV misclassification rate and fold SE, all candidates (best to worst):
      pcalda-6  pcalda-5  pcaqda-5  pcalda-7  pcalda-8      lda  pcaqda-6
cv_error    0.2513    0.2542    0.2545    0.2598    0.2713    0.2742    0.2747
se          0.0168    0.0197    0.0191    0.0149    0.0143    0.0149    0.0180
      pcalda-4  pcaqda-7  blend-0.75  pcaqda-8  pcaqda-4  blend-0.5  pcaqda-2
cv_error    0.2831    0.2836    0.2859    0.2948    0.2951    0.3006    0.3008
se          0.0194    0.0222    0.0169    0.0253    0.0233    0.0206    0.0188
      pcaqda-3      qda  pcalda-3  blend-0.25  pcaqda-1  pcalda-1  pcalda-2
cv_error    0.3121    0.3182    0.3207    0.3211    0.3211    0.3240    0.3355
se          0.0174    0.0247    0.0220    0.0241    0.0223    0.0191    0.0219
selected: pcalda-6
test misclassification rate of pcalda-6: 0.2759 (95% Wilson interval 0.203 to 0.363, n = 116)
variance explained by the kept components: [0.317 0.133 0.118 0.108 0.093 0.084] total 0.853
priors: [0.6532 0.3468]
class means on the component scale (rows chd 0, 1):
[[-0.449 -0.032 -0.118 -0.113 -0.147 -0.044]
 [ 0.845  0.06  0.222  0.213  0.277  0.082]]
```

(c) Review and revision. I checked the agent's run against the skill before looking at the test line: the folds are built once and reused, the scaler and PCA are refitted inside every fold, and the test set appears in exactly one `posterior` call. One revision came out of the first run. The first version of the skill asked only for the mean CV error, and the sorted list made the top candidates look clearly separated when the fold standard errors (about 0.017) were as large as the gaps between them. I added the requirement to report the fold SE with every CV error and to say plainly when the winner is within the noise of plain LDA; that is the version appended below, and it was decided from training and CV output only. The file is 4,205 bytes. The one-page analysis report follows, exactly as the skill lays it out.

Analysis report (one page)

SAheart, response `chd` , 9 predictors. Train n = 346, test n = 116, seed 4327.

Candidates and procedure. All classifiers are LDA, QDA, or a combination: LDA and QDA on the raw predictors, LDA and QDA on the first k principal components of the standardized predictors for k = 1 to 8, and a blend of the LDA and QDA posteriors with weight w on LDA. A stratified 25 percent test set was set aside first. The 21 candidates were compared on the same ten stratified folds of the training set, with standardizing and PCA refitted on the training part of each fold, using the misclassification rate at the 0.5 posterior cutoff.



candidate				
	role	cv_error	se	parameters
PCA(6)+LDA	selected	0.251	0.017	33
LDA	comparison	0.274	0.015	63
QDA	comparison	0.318	0.025	108
PCA(5)+LDA	next best	0.254	0.020	25
PCA(5)+QDA	next best	0.254	0.019	40

Selected model and test error. The lowest CV error is 0.251 (SE 0.017) for PCA(6)+LDA, so that model was refitted on all 346 training rows; the six components keep 85.3% of the standardized variance and the fitted priors are 0.653 and 0.347. The class means on the component scale are printed above. Its single evaluation on the reserved test set gives a misclassification rate of **0.276** (32 of 116; 95 percent Wilson interval 0.203 to 0.363).

Findings. The data favour a shared covariance: every LDA-type candidate beats its QDA counterpart at the same dimension, plain QDA (0.318) is the second worst raw-predictor model, and blending toward QDA only makes LDA worse. Dropping the last three principal components lowers the CV error of LDA from 0.274 to 0.251, but that gap is about one fold SE, so dimension reduction helps at most slightly. The test error of 0.276 is in line with the CV estimates for all of the top candidates rather than confirming a real gain over LDA.

Limitations. With 116 test cases the interval on the test error is about 16 percentage points wide, so it cannot separate models that differ by two or three points. All candidates were ranked on the same folds, so the minimum CV error is optimistically biased as an estimate of the selected model's error, which is why the test set is reported separately. Several predictors (`tobacco` , `alcohol` , `famhist`) are far from normal or are coded 1/2, so the normal-likelihood assumption behind LDA and QDA is only an approximation here.

Appendix: complete `SKILL.md`

Printed from the file as submitted; the size check is computed from the same file.

```
name: lda-qda-report
description: Run a discriminant-analysis study (LDA, QDA, PCA+LDA/QDA, LDA-QDA blend) on a user-supplied dataset and write a one-page statistical report. Only runs when invoked explicitly with /lda-qda-report.
disable-model-invocation: true
argument-hint: data=<file> response=<column> predictors=<columns or "all others"> seed=<int> [test_frac=0.25] [fold s=10]
```

lda-qda-report

You are a careful statistician. The user supplies the dataset, the response column, the predictor columns (with any coding), and a seed in \$ARGUMENTS. Nothing about the data is assumed by this skill. Ask once if any of the four inputs is missing; otherwise do not ask questions, just run the study.

Behavior

1. Reproducibility first. Set the seed before the split and reuse it for the folds. Report the seed, the software versions, and the sample sizes.
2. The test set is touched once. Reserve a stratified test set (default 25%) before looking at any model. Every comparison and every setting is decided by cross-validation inside the training set only.
3. Only LDA, QDA, or combinations of them are allowed as classifiers.
4. Any preprocessing (standardizing, PCA) is fitted on the training part of each fold and applied to the held-out fold, never on the whole data.
5. Say what you did, not what you hoped. If the selected model is not better than plain LDA by more than the fold-to-fold noise, say so.

Analysis steps

1. Load the data, apply the user's coding, drop rows with missing values and report how many were dropped.
2. Stratified split: train / test with the given fraction.
3. Stratified K-fold CV (default K=10, shuffled with the seed) on the training set. Use the same folds for every candidate.
4. Candidates, all with empirical class priors and the 0.5 posterior cutoff:
 - LDA on the raw predictors; QDA on the raw predictors.
 - PCA + LDA and PCA + QDA: standardize inside the fold, keep the first k components for $k = 1 \dots p-1$, then fit.
 - Blend: $\text{posterior} = w * \text{LDA} + (1 - w) * \text{QDA}$ for w in $\{0.25, 0.5, 0.75\}$.
5. Score each candidate by mean CV misclassification rate and its standard error across folds ($\text{sd} / \sqrt{K}$).
6. Select the candidate with the lowest CV error. Ties within 0.001 go to the simpler model (fewer components, no blend, LDA before QDA).
7. Refit the selected preprocessing and model on all training rows and evaluate once on the test set. Report the test misclassification rate with a 95% binomial (Wilson) interval.

Report (at most one printed page, letter size)

Produce it as a single section with these headings in this order. Plain sentences, first person, no bullet lists longer than four items, no code.

1. ****Title line****: dataset name, response, n train / n test, seed.
2. ****Candidates and procedure****: two or three sentences on the model family, the split, the CV design, and where preprocessing is fitted.
3. ****Figure****: one figure, 6.5 x 2.8 inches, CV error (with ± 1 SE bars) for every candidate, sorted from best to worst, candidate names on the x axis, PCA candidates labeled by k. Okabe-Ito colors, no gridlines.
4. ****Table****: one table with the selected model, plain LDA, plain QDA, and the next two best candidates. Columns: candidate, CV error, SE, number of fitted parameters (means + covariance entries). Round to 3 decimals.
5. ****Selected model and test error****: the selection, its refit on the full training set, the single test error with its interval, and the numerical fitting results (priors, class means on the model scale, and for PCA the variance explained by the kept components).
6. ****Findings****: three or four sentences. Which assumption (shared or separate covariance) the data support, whether dimension reduction or blending helped beyond the fold noise, and what the test error says.
7. ****Limitations****: two or three sentences, including the width of the test interval, the fact that candidates were compared on the same folds, and anything about the data that makes LDA/QDA assumptions doubtful.

Keep every number in the report traceable to printed output. Do not report a number you did not compute in this run.